



ASSIGNMENT TITLE :

General Report on Investing Process Lifecycle and OS Interactions(Lab_3)

Module Code: ST5003CMD

Module Name: Operating Systems, Security, and Networks

Intake: March 2025

Submission date: September 26th,2026

Submitted By

Name : Ujwal Acharya
Student ID : 16537235
College ID : 250036

Submitted To

Rupak Rajbanshi
Lecturer

Softwarica College of IT & E-Commerce
In Collaboration with Coventry University
Kathmandu, Nepal

Table of Contents

Table of Figures.....	3
1. Task 1: The Long-Running Process	4
2. Task 2: Process identity (PID and PPID)	6
3. Task 3: Exit codes and OS feedback	8
4. Task 4:Standard I/O Streams.....	10
5. Task 5: Conditional execution and termination	11
6. Conclusion	13

Table of Figures

Figure 1: Creating task1.c file	4
Figure 2: task1.c code.....	4
Figure 3: Terminal 1st ,code execution	5
Figure 4: Terminal 2nd ,Process verification.....	5
Figure 5: Creating task2.c using nano	6
Figure 6: task2 code to get PID and PPID	6
Figure 7: Output of task2.c	7
Figure 8: Verify the PID and PPID in another terminal	7
Figure 9: Creating Task3.c file using nano	8
Figure 10: Code inside task3.c	8
Figure 11: Output of task3.c if a positive number is entered	9
Figure 12: Output of task3.c if a negative number is entered	9
Figure 13: Creating task4.c using nano.....	10
Figure 14: Code inside task4.c	10
Figure 15: Execution of task4.c.....	10
Figure 16: Creating task5.c using nano.....	11
Figure 17: Code of task5.c	11
Figure 18: Output of task5.c when entering 1	12
Figure 19: Output of task5.c when entering 0.....	12

1. Task 1: The Long-Running Process

OS insights:

In enterprise servers, processes run for days or months. We must know how to monitor them while they are active.

a) C source Code :

First, we created a folder Week-3, and then inside that we created lab_3, where a codes folder is created specifically to hold all the task codes written on VirtualBox Ubuntu. Inside the code folder, task1.c is created.

```
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks$ mkdir Week-3
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks$ cd Week-3
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3$ mkdir Lab_3
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3$ cd Lab_3
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3$ mkdir codes
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3$ cd codes
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$ nano task1.c
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$
```

Figure 1: Creating task1.c file

```
GNU nano 8.7.1 task1.c *
#include <stdio.h>
#include <unistd.h>

int main(){
    printf("I am starting...\n");

    for(int i=1; i <=30; i++){
        sleep(1); //pauses execution for 1 second
    }

    printf("I am finished.\n");
    return 0;
}
```

Figure 2: task1.c code

b) Terminal Execution and Verification:

```
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$ gcc task1.c -o task1
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$ ./task1
I am starting...
I am finished.
```

Figure 3: Terminal 1st ,code execution

```
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$ ps aux | grep task1
ujwal06+ 12879 0.0 0.0 2768 1756 pts/0 S+ 07:48 0:00 ./task1
ujwal06+ 12891 0.0 0.0 9316 2676 pts/1 S+ 07:48 0:00 grep --color=auto task1
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$
```

Figure 4: Terminal 2nd ,Process verification

This task demonstrates how a process can remain active for a specific period. The task1.c uses a loop and sleep() to run for 30 seconds, and during this period, on another terminal, the process is monitored using the **ps aux | grep task1** command.

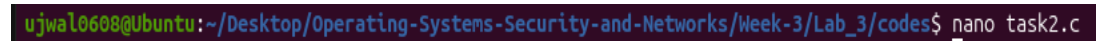
2. Task 2: Process identity (PID and PPID)

OS Insight:

Every process in Linux has a unique Process ID (PID). The process that created it is the Parent Process ID (PPID), usually our terminal shell.

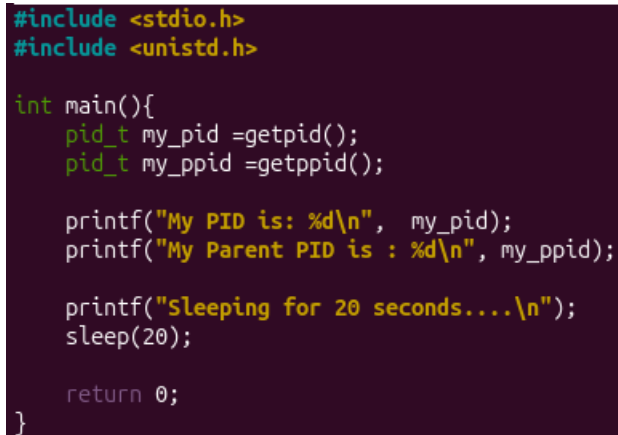
a) C Source code:

In this, the task2.c file is created to print the unique process ID (PID) and parent process ID (PPID).

A terminal window with a dark background. The prompt is 'ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes\$'. The command 'nano task2.c' has been entered, and a cursor is visible at the end of the line.

```
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$ nano task2.c
```

Figure 5: Creating task2.c using nano

A screenshot of the C source code for task2.c. The code is displayed in a dark-themed editor with syntax highlighting. It includes headers for stdio.h and unistd.h, defines a main function that gets the current process ID (PID) and parent process ID (PPID), prints them, sleeps for 20 seconds, and then returns 0.

```
#include <stdio.h>
#include <unistd.h>

int main(){
    pid_t my_pid =getpid();
    pid_t my_ppid =getppid();

    printf("My PID is: %d\n", my_pid);
    printf("My Parent PID is : %d\n", my_ppid);

    printf("Sleeping for 20 seconds....\n");
    sleep(20);

    return 0;
}
```

Figure 6: task2 code to get PID and PPID

b) Terminal Execution and Verification :

```
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$ gcc task2.c -o task2
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$ ./task2 &
[1] 13347
My PID is: 13347
My Parent PID is : 5008
Sleeping for 20 seconds....
```

Figure 7: Output of task2.c

```
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$ ps -p 13347 -o pid,ppid,cmd
  PID   PPID  CMD
  13347   5008  ./task2
```

Figure 8: Verify the PID and PPID in another terminal

In general, this task shows the PID and PPID. PID uniquely identifies a process, while PPID identifies the parent process that created it. We get the PID and PPID from one terminal, and then in another terminal, we use **ps -p pid -o pid,ppid,cmd** to verify the running process and its ID.

3. Task 3: Exit codes and OS feedback

OS insights:

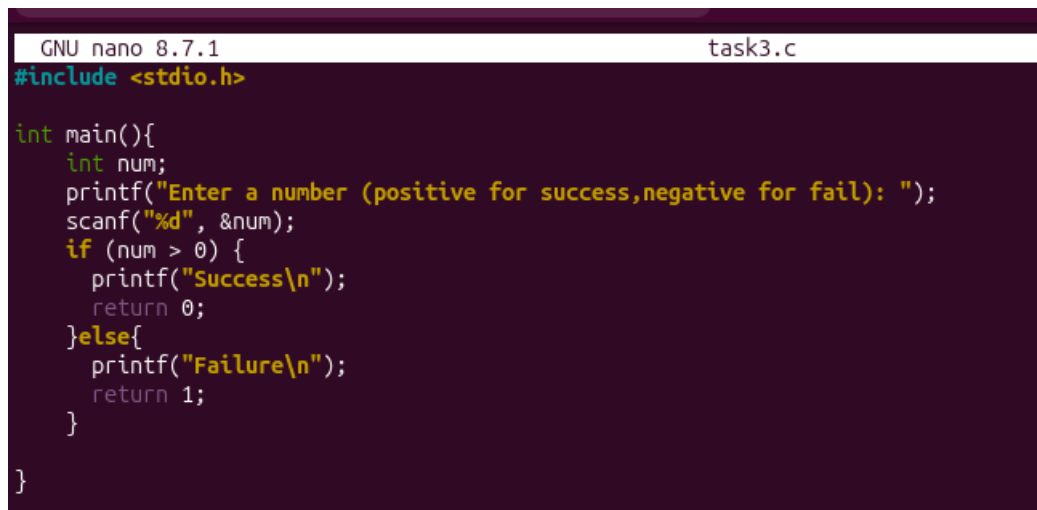
When a program finishes, it returns an integer to the OS .0 means “success”, and any non-zero number (like 1) means “error/failure”. The shell stores this in \$?

A) C Source code :

In this, task3.c is created to print the entered value and show whether it returns 1 or 0. If the program runs successfully, it will return 0, and if not, it returns 1.

```
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$ nano task3.c
```

Figure 9: Creating Task3.c file using nano



```
GNU nano 8.7.1 task3.c
#include <stdio.h>

int main(){
    int num;
    printf("Enter a number (positive for success,negative for fail): ");
    scanf("%d", &num);
    if (num > 0) {
        printf("Success\n");
        return 0;
    }else{
        printf("Failure\n");
        return 1;
    }
}
```

Figure 10: Code inside task3.c

B) Terminal Execution and Verification :

```
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$ gcc task3.c -o task3
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$ ./task3
Enter a number (positive for success,negative for fail): 5
Success
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$ echo $?
0
```

Figure 11: Output of task3.c if a positive number is entered

```
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$ ./task3
Enter a number (positive for success,negative for fail): -5
Failure
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$ echo $?
1
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$
```

Figure 12: Output of task3.c if a negative number is entered

This task shows how a program communicates its status to the OS. When 0 is returned, it represents successful execution, while a non-zero return value represents an error or failure.

4. Task 4:Standard I/O Streams

OS Insights:

The OS provides standard output (stdout) streams. `scanf()` reads from stdin, and `printf()` writes to stdout.

a) C Source code :

In this,task4.c is created, which accepts input values using stdin and demonstrates the output using stdout.

```
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$ nano task4.c
```

Figure 13: Creating task4.c using nano



```
GNU nano 8.7.1 task4.c *
#include <stdio.h>

int main(){
    char name[50];

    printf("Enter your name: ");
    scanf("%s", name);

    printf("Hello, %s! Welcome to OS class.\n",name);

    return 0;
}
```

Figure 14: Code inside task4.c

b) Terminal Execution :

```
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$ gcc task4.c -o task4
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$ ./task4
Enter your name: Ujwal
Hello, Ujwal! Welcome to OS class.
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$
```

Figure 15: Execution of task4.c

This task shows the use of Linux standard input (stdin) and standard output (stdout). The C program uses `scanf()` to get input values from the user and `printf()` to display the output.

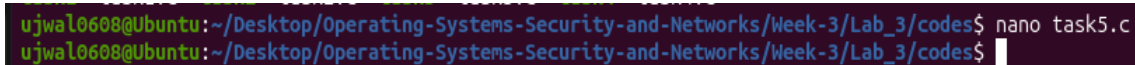
5. Task 5: Conditional execution and termination

OS insights:

Programs often branch on user input. The OS tracks the final exit code to determine if the script or automation pipeline should continue or halt.

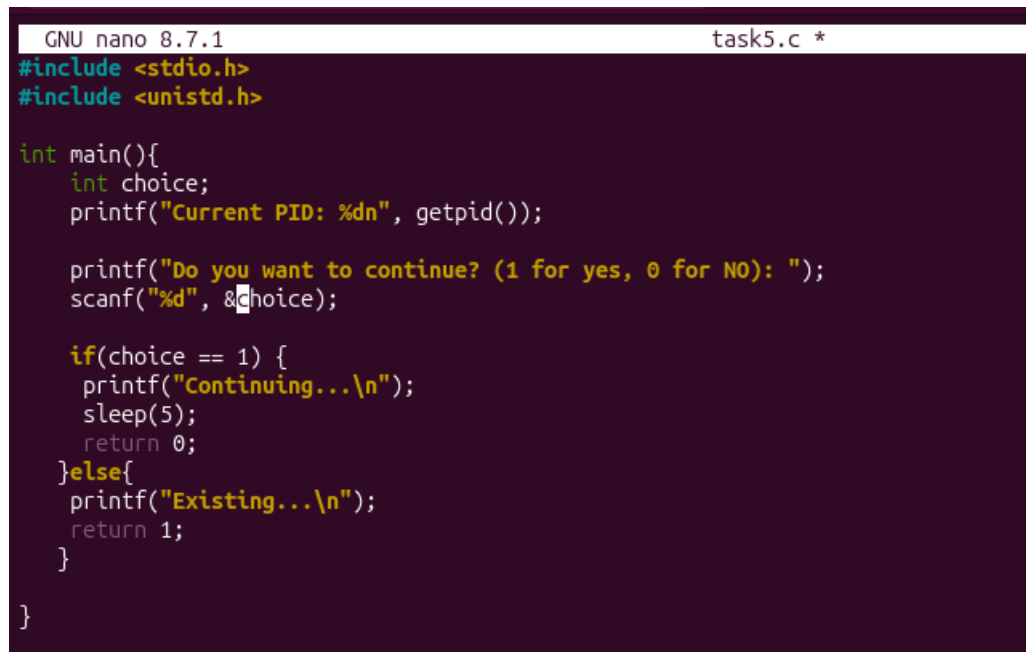
a) C Source code:

At first, task5.c is created, and in it, it displays the PID and asks whether to continue or not, taking user input 1 or 0.



```
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$ nano task5.c
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$
```

Figure 16: Creating task5.c using nano



```
GNU nano 8.7.1 task5.c *
#include <stdio.h>
#include <unistd.h>

int main(){
    int choice;
    printf("Current PID: %dn", getpid());

    printf("Do you want to continue? (1 for yes, 0 for NO): ");
    scanf("%d", &choice);

    if(choice == 1) {
        printf("Continuing...\n");
        sleep(5);
        return 0;
    }else{
        printf("Existing...\n");
        return 1;
    }
}
```

Figure 17: Code of task5.c

b) Terminal Execution and Verification:

```
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$ gcc task5.c -o task5
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$ ./task5
Current PID: 16525nDo you want to continue? (1 for yes, 0 for NO): 1
Continuing...
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$ echo $?
0
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$
```

Figure 18: Output of task5.c when entering 1

```
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$ ./task5
Current PID: 16529nDo you want to continue? (1 for yes, 0 for NO): 0
Existing...
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$ echo $?
1
ujwal0608@Ubuntu:~/Desktop/Operating-Systems-Security-and-Networks/Week-3/Lab_3/codes$
```

Figure 19: Output of task5.c when entering 0

This task shows how a program makes decisions based on user input and terminates with an appropriate exit code.

6. Conclusion

In conclusion, these tasks give a practical understanding of how the Linux operating system manages process and program execution. These tasks demonstrate the process lifetime, **PID** and **PPID**, exit codes, standard input and standard output, and conditional program termination.